HierFinRAG & TableTransformerRAG: Hierarchical and Vision-Based Retrieval-Augmented Generation for Complex Financial Document Question Answering

Aravind
School of Computer and Information Sciences
University of Hyderabad, Hyderabad, India
Work conducted during Summer Research Internship at IIT Hyderabad

***Abstract*—Financial document question answering (QA) over large corporate annual reports demands precision that naïve flat Retrieval-Augmented Generation (RAG) systems cannot deliver: tables must be parsed with structural fidelity, arithmetic must be exact, and the system must handle long (>100 page) PDFs without hallucinating numbers. We present two advanced RAG architectures benchmarked on a 110-question FinQA-style dataset drawn from the Apple 2023 and Alphabet 2025 10-K filings. HierFinRAG constructs a four-level document chunk tree (section, text, table, cell) and retrieves via a three-level pipeline combining BM25 sparse search, BGE-M3 dense embeddings, Reciprocal Rank Fusion (RRF), and a cross-encoder reranker. A symbolic Domain-Specific Language (DSL) calculator evaluates arithmetic deterministically to eliminate LLM hallucinations, achieving 40.0% execution accuracy and 93.4% retrieval recall. TableTransformerRAG replaces heuristic PDF table parsing with Microsoft's Table Transformer (TATR), a DETR-based vision model that detects and parses tables directly from 300-DPI rendered page images. After four iterative engineering rounds addressing TATR bounding-box mapping errors, context overload, and a two-pass extraction strategy, TableTransformerRAG achieves 48.2% execution accuracy and 89.7% recall—surpassing HierFinRAG on accuracy by 8.2 percentage points. Both systems run fully locally via Ollama using open-weight models, demonstrating that high-quality financial document intelligence is achievable without commercial cloud infrastructure.**

***Index Terms*—Retrieval-Augmented Generation, Financial NLP, Hierarchical Retrieval, Table Detection, DETR, Symbolic Reasoning, Question Answering, Large Language Models**

I. Introduction

Corporate annual reports (Form 10-K filings) represent one of the most challenging document types for automated question answering. They span hundreds of pages, intermix dense prose with numerical tables, and require answering questions that necessitate both exact factual retrieval and precise multi-step arithmetic. Financial analysts must navigate cross-sectional dependencies, fiscal year terminology (e.g., "September 24, 2022" referring to FY2022), and magnitude conventions ($millions vs. $billions) that change silently across pages.

Standard RAG pipelines—which chunk documents uniformly, embed chunks with a dense encoder, and retrieve the top-k by cosine similarity—fail catastrophically in this domain. In our evaluation, a flat baseline (BasicRAG) achieves only **3.0%** execution accuracy (EM) despite 80.3% retrieval recall, confirming that the bottleneck is structural understanding and arithmetic reasoning, not retrieval coverage alone.

This paper makes the following contributions:

1) We design and evaluate **HierFinRAG**, a three-level hierarchical retrieval system with intent routing, BM25+dense hybrid search, Reciprocal Rank Fusion (RRF), and a symbolic FinQA DSL calculator—achieving 40.0% EM and 93.4% retrieval recall on a 110-question benchmark.
2) We design **TableTransformerRAG**, which replaces heuristic PDF parsing with DETR-based visual table detection (Microsoft Table Transformer) and achieves 48.2% EM after four iterative engineering rounds.
3) We document 13 engineering bugs and their fixes across the full development cycle, providing a rigorous failure analysis and lessons for practitioners building financial RAG systems.
4) We demonstrate a **retrieval-generation paradox**: increasing retrieval context for 7B models beyond a threshold degrades generation quality, requiring intent-conditioned context sizing.

All experiments are conducted entirely locally using Ollama with Qwen 2.5-7B, Mistral 7B, and LLaVA 7B—no commercial API is required.

II. Related Work

A. Retrieval-Augmented Generation

RAG [1] augments LLM generation with retrieved evidence, reducing hallucinations on knowledge-intensive tasks. Standard dense retrieval (e.g., DPR [2]) encodes documents and queries into a shared vector space and retrieves by maximum inner product. Hybrid retrieval [3] fuses sparse (BM25) and dense signals, consistently outperforming either alone on heterogeneous document corpora. Reciprocal Rank Fusion [4] provides a score-free merging strategy that is robust to ranking calibration mismatches.

B. Financial Document Understanding

FinQA [5] introduced the multi-step financial reasoning challenge: given a table and supporting text, answer numerical questions using a structured DSL program evaluated by an interpreter. Our symbolic calculator is inspired by this formulation but adapts it for open-domain retrieval over full 10-K corpora rather than pre-segmented gold contexts. TAT-QA [6] further motivates hybrid table-text reasoning. HIBRIDS [7] studies hierarchical document summarization in financial contexts.

C. Visual Table Parsing

Table detection and structure recognition are fundamental to document AI. The Table Transformer (TATR) [8] fine-tunes DETR [9] on PubTables-1M for table detection and structure recognition, achieving state-of-the-art on academic document benchmarks. We adapt TATR to financial PDF pages rendered at 300 DPI, including a novel coordinate mapping from crop-image pixel space to PDF-point space.

D. Local LLM Inference

Recent open-weight models such as Qwen 2.5 [10] and Mistral [11] enable high-quality inference on consumer hardware. Ollama provides an OpenAI-compatible local serving API. Our work evaluates these models in a challenging structured reasoning scenario and characterises their context-length sensitivity.

III. Problem Formulation & Dataset

A. Task Definition

Given a natural language question q and a corpus of financial PDF documents $\mathcal{D}$, the system must produce an answer $\hat{a}$ such that $\hat{a} \approx a^*$, the ground-truth answer. Questions require either (i) factual lookup of a specific value in a table cell or paragraph, or (ii) multi-step arithmetic over retrieved values (e.g., percentage change between two fiscal years). We define four intent categories:

- **Numerical** ($n = 50$): Arithmetic over retrieved values (e.g., percentage change, ratios).
- **Lookup** ($n = 50$): Direct factual extraction of a specific value from a table or text.
- **Comparison** ($n = 5$): Qualitative comparison across periods or segments.
- **Summarization** ($n = 5$): Open-ended explanation or description.

B. Evaluation Dataset

We construct a 110-question FinQA-style benchmark drawn from:

- **Apple Inc. 2023 Form 10-K** (55 questions): A clean, well-structured PDF with native bookmarks but date-encoded column headers (e.g., "September 24, 2022") that cause systematic LLM year-mapping errors.
- **Alphabet Inc. 2025 Form 10-K** (55 questions): A larger document with *no* internal PDF bookmarks, causing zero section nodes to be detected by heuristic parsers, requiring a special fallback strategy.

Each question record contains: `question`, `answer`, `intent`, `source` (PDF filename), and optional FinQA gold context fields (`pre_text`, `table_text`, `post_text`). The gold context is used *exclusively* to compute Retrieval Recall and is never passed to the LLM generator.

C. Evaluation Metrics

- **Execution Accuracy (EM)**: Exact-match with 1% relative tolerance and bidirectional scale reconciliation (decimal/percentage, millions/billions).
- **Scaled F1**: Token-level overlap in $[0, 1]$.
- **Retrieval Recall**: Fraction of gold context chunks present in the top-k retrieved set (keyword overlap matching).
- **Latency**: Total wall-clock seconds for all 110 questions.

IV. System Architecture

A. BasicRAG (Flat Baseline)

BasicRAG is a minimal RAG baseline with no structural awareness: `pdfplumber` extracts text and tables as flat chunks; BGE-M3 encodes them into Qdrant; top-k cosine retrieval feeds raw context to the LLM. Despite 80.3% retrieval recall, BasicRAG achieves only 3.0% EM, confirming that retrieval coverage alone does not produce accurate financial QA.

B. HierFinRAG

1) Hierarchical Document Parser: The parser (`pipeline/parser.py`) builds a four-level chunk tree: **Section** nodes (short, title-cased or ALL-CAPS headings), **Text** paragraphs, **Table** chunks (with `col_headers`, `row_headers`, and `rows`), and **Cell** chunks (one per row×column intersection, formatted as "*row — col: value*"). Each chunk carries a globally unique `chunk_id`, `parent_id`, and `children_ids`, forming a navigable document graph.

2) Intent Router: A two-stage classifier (`pipeline/router.py`) routes each query into one of four intent classes. Stage 1 applies rule-based weighted keyword scoring over four vocabulary sets augmented by math-operator pattern matching (fast, no LLM call). Stage 2 invokes a single-token Ollama call as a fallback when Stage 1 scores are zero or tied.

3) Three-Level Retrieval: **Level 1** applies metadata source filtering (preventing cross-document retrieval) and query expansion for numerical/comparison intents: year tokens matching four-digit year patterns (e.g., `r"\b(20\d{2})\b"`) and financial keywords are appended to shift query embeddings toward tabular content.

Level 2 executes hybrid BM25 (`rank_bm25.BM25Okapi`) + BGE-M3 dense search over section children, fused with Reciprocal Rank Fusion:

$$\text{RRF}(d) = \sum_{i \in \{\text{BM25, dense}\}} \frac{1}{k + \text{rank}_i(d)}, \quad k = 60. \tag{1}$$

A cross-encoder reranker (`BAAI/bge-reranker-v2-m3`) rescores the top-N candidates.

Level 3 runs dense matching against individual cell chunks (`type=''cell''`) to isolate the exact row-column intersection for numerical queries.

4) Symbolic-Neural Fusion Generator: To eliminate arithmetic hallucinations, the generator (`pipeline/generator.py`) prompts the LLM to produce a FinQA DSL program rather than computing the answer directly:

$$\texttt{percentage\_change}(v_{\text{old}}, v_{\text{new}}) = \frac{v_{\text{new}} - v_{\text{old}}}{v_{\text{old}}} \quad (2)$$

The `SymbolicCalculator` module (`pipeline/symbolic.py`) evaluates the program deterministically via `numexpr`, supporting nested expressions up to depth 15 (`add`, `subtract`, `multiply`, `divide`, `percentage_change`, `percentage`). This decouples arithmetic correctness from LLM generation quality.

C. TableTransformerRAG

1) Vision-Based Table Parsing: Instead of heuristic `pdfplumber` table extraction, each PDF page is rendered to a PIL Image at 300 DPI using PyMuPDF (`fitz`). The TATR detection model (`microsoft/table-transformer-detection`) identifies table bounding boxes (confidence threshold = 0.85). Each detected region is cropped (with 5 px padding) and passed to the TATR structure recognition model (`microsoft/table-transformer-structure-recognition-v1.1-all`), which outputs row, column, and header bounding boxes in crop-pixel coordinates.

2) Coordinate Mapping: Cell bounding boxes are mapped from crop-pixel space to PDF-point space using the actual PIL crop dimensions (w_c, h_c) and the table's PDF-point extent (x_1, y_1, x_2, y_2):

$$x_{\text{pdf}} = x_1 + \frac{x_{\text{px}}}{w_c}(x_2 - x_1), \quad (3)$$

$$y_{\text{pdf}} = y_1 + \frac{y_{\text{px}}}{h_c}(y_2 - y_1). \quad (4)$$

Word objects (native PDF text in PDF-point coordinates) are matched to cells by centre-point containment, recovering accurate text without OCR.

3) Non-Table Text Recovery: Words not covered by any detected table bounding box are grouped by y-coordinate proximity into lines, merged into paragraphs, and split at 500 words. This produces text chunks structurally identical to HierFinRAG, enabling a fair downstream comparison.

4) Retrieval and Generation: TableTransformerRAG shares the same three-level retrieval, RRF hybrid search, cross-encoder reranker, intent router, and symbolic DSL calculator as HierFinRAG. The only difference is table chunk provenance (visual TATR detection vs. heuristic pdfplumber parsing).

V. Engineering Challenges & Iterative Optimization

Developing reliable financial RAG systems required identifying and resolving 13 distinct failure modes across the parsing, retrieval, generation, and evaluation stages. We describe the most impactful ones below.

A. Evaluation Integrity: Gold Context Leakage (Bug 1)

The FinQA dataset provides a "gold context" (pre_text, table_text, post_text) alongside each question. In an early implementation, this gold context was passed directly to the LLM generator, bypassing retrieval entirely. This inflated apparent accuracy to $\sim$70%; the true EM after isolation was 3.0% for BasicRAG. **Fix:** `evaluate.py` was rewritten to pass only the raw question and source filter to the retrieval pipeline; gold context is used solely to compute Retrieval Recall.

B. Alphabet 10-K: Zero Section Nodes (Bug 6)

Alphabet's 2025 10-K headings failed the heuristic `_is_header()` test (short, title-cased, no trailing period), producing zero section chunks. Level 1 retrieval, which searches over section nodes, returned zero results for all Alphabet queries, driving recall to 0%.

Fix (Alphabet Section Fallback): After parsing, if section count is zero, all text chunks are promoted to `type=''section''` and linked to adjacent table chunks as children:

$$\text{children}(c_i) = \bigcup_{p \in \{pg-1, pg, pg+1\}} \text{tables}(p) \quad (5)$$

where pg is the page of text chunk c_i.

C. TATR Bounding-Box Coordinate Error (Bug 13)

TATR structure model outputs coordinates in crop-image-pixel space. The initial implementation mapped these using the full-page scale factor, causing systematic cell offset errors and empty text extractions. **Fix:** Equations (3)–(4) use the actual PIL crop dimensions, resolving the offset.

D. The Retrieval-Generation Paradox (Bug 8)

In Round 3, increasing `top_n` from 10 to 15 raised Retrieval Recall from 69.2% to 89.7%, but EM dropped from 38.2% to 36.4%. A 7B parameter model with 15 competing financial fragments showed attention divergence, producing narrative prose instead of precise value extraction.

Fix (Intent-Conditioned Context Sizing): `top_n` is set by intent at query time: Lookup = 10, Summarization = 15, and Numerical = 10 with table cell relevance scores boosted by $+3.0$ above narrative text to force tabular figures to the top of the LLM context window.

E. Lookup Narrative Rambling (Bug 12)

For factual lookup queries, the 7B LLM answered correctly but embedded the answer inside multi-sentence prose. Automated exact-match evaluation failed because no clean extracted value was found. 94% of baseline Lookup questions scored zero.

Fix (Two-Pass Extraction): If a Lookup answer exceeds 50 characters, a second zero-shot extraction call is made with the directive: *"Give ONLY the exact value that answers the*

question. At most 10 words. No sentences, no explanation." This decouples reasoning quality from output format adherence and raised Lookup accuracy from 6.0% to 54.0% (+800%).

F. TableTransformerRAG Iterative Optimization Summary

Table I summarises the four engineering rounds:

TABLE I
TABLETRANSFORMERRAG OPTIMIZATION ACROSS FOUR ROUNDS

Stage	EM (%)	F1	Recall (%)
Baseline (R0)	26.4	0.274	37.5
Round 1	34.5	0.267	80.1
Round 2	38.2	0.332	69.2
Round 3	36.4	0.282	**89.7**
Round 4 (Final)	**48.2**	**0.387**	**89.7**

Round 1 raised the TATR detection threshold ($0.70 \rightarrow 0.85$) and normalised BM25 tokenisation; Recall: $37.5\% \rightarrow 80.1\%$. **Round 2** added row-signature deduplication, heuristic header promotion, and the Alphabet section fallback; Comparison accuracy: $0\% \rightarrow 60\%$. **Round 3** injected section text payload into Level-2 candidates; Recall: 89.7% (peak), but EM dipped due to context overload. **Round 4** applied intent-conditioned context sizing, two-pass extraction, date-column mapping directives, and bidirectional scale reconciliation; EM: 48.2% (peak).

VI. EXPERIMENTAL RESULTS

A. HierFinRAG: LLM Backend Comparison

Table II compares all four HierFinRAG configurations on 110 QA pairs. All share identical retrieval recall (93.4%), confirming the bottleneck is generation quality, not retrieval coverage. Qwen 2.5-7B achieves the highest EM (40.0%). LLaVA 7B adds +5,505 s latency for no accuracy gain, as vision-language models are ill-suited to text-only financial QA. Qdrant is ≈47% faster than ChromaDB at identical accuracy.

TABLE II
HIERFINRAG: LLM AND VECTOR-STORE CONFIGURATION COMPARISON (110 QA)

Config	EM	F1	Rec.	Lat. (s)
Qwen 2.5 + Qdrant	**0.400**	0.240	**0.934**	**8,097**
Mistral 7B + Qdrant	0.318	0.155	**0.934**	8,878
LLaVA 7B + Qdrant	0.273	0.167	**0.934**	13,602
Qwen 2.5 + Chroma	**0.400**	**0.248**	**0.934**	11,966

B. Cross-System Comparison

Table III compares the best configuration of each system. TableTransformerRAG (Round 4) achieves the highest EM and F1. HierFinRAG achieves slightly higher retrieval recall due to its reliable heuristic section detection on Apple (which has proper PDF bookmarks). BasicRAG, despite strong recall, fails on generation—confirming that structure-aware retrieval and symbolic arithmetic are both necessary.

TABLE III
CROSS-SYSTEM BEST-CONFIGURATION COMPARISON (110 QA PAIRS)

System	EM	F1	Recall	Lat. (s)
BasicRAG	0.030	0.104	0.803	1,375
HierFinRAG	0.400	0.240	**0.934**	8,097
TATR-R4	**0.482**	**0.387**	0.897	≈4,800

C. Per-Intent Analysis

Table IV shows route-level accuracy. TableTransformerRAG-R4 outperforms HierFinRAG on Lookup (54.0% vs. 38.0%) and equals it on Summarization (60.0%). Both systems score 0/5 on Comparison without the evaluator fix (Round 2); after the fix, TATR-R4 achieves 3/5 (60.0%). HierFinRAG shows stronger Numerical performance (22/50 vs. 20/50) due to more reliable table structure from pdfplumber on Apple's clean tables.

TABLE IV
PER-INTENT EXECUTION ACCURACY: BEST CONFIGURATION PER SYSTEM

System	Num.	Lookup	Cmp.	Sum.
BasicRAG	3/50	0/50	–	–
HierFinRAG	22/50	19/50	0/5	3/5
TATR-R4	20/50	**27/50**	**3/5**	**3/5**

D. Per-Document Analysis

Table V shows breakdown by document. HierFinRAG achieves equal EM on both documents (40.0%), reflecting consistent retrieval. TATR-R4 scores higher on Alphabet (56.4% vs. 40.0%) but slightly lower on Apple, indicating that visual table detection better handles Alphabet's bookmark-free structure once the section fallback is active.

TABLE V
PER-DOCUMENT BREAKDOWN: BEST CONFIG PER SYSTEM

System	Apple 2023		Alphabet 2025	
	EM	Rec.	EM	Rec.
BasicRAG	0.040	0.689	0.020	0.917
HierFinRAG	0.400	**0.931**	0.400	**0.937**
TATR-R4	0.400	0.901	**0.564**	0.893

VII. DISCUSSION

A. Retrieval-Generation Paradox

Our experiments reveal a fundamental tension in deploying 7B parameter models for structured information retrieval. While retrieval recall saturated at 89.7% in Round 3, execution accuracy simultaneously declined. This *retrieval- generation paradox* arises because small language models have limited attention capacity: providing 15 dense financial chunks overloads the context window, causing the model to produce verbose narrative responses rather than precise numerical extraction. Intent-conditioned context sizing (Section V-D) resolves this

by matching context volume to the model's capacity for each query type.

B. Symbolic Arithmetic vs. LLM Generation

The symbolic DSL calculator is the single most impactful design decision in both HierFinRAG and TableTransformerRAG. Directly asking a 7B LLM to compute $(394{,}328 - 383{,}285)/383{,}285$ yields unreliable results due to tokenisation of large numbers and arithmetic hallucination. The DSL approach separates symbolic program synthesis (LLM's strength) from exact arithmetic evaluation (`numexpr`'s strength), achieving deterministic numerical correctness.

C. Vision-Based vs. Heuristic Table Parsing

TATR outperforms pdfplumber on Alphabet's unstructured PDF (56.4% vs. 40.0% EM) but is approximately equal on Apple's clean document (40.0% vs. 40.0%). This suggests that vision-based detection is most beneficial for documents with non-standard formatting, mixed layouts, or absent PDF bookmarks. The 300 DPI rendering requirement and CPU/GPU inference overhead make TATR slower per-document during indexing, but its superior table boundary detection justifies this cost for complex financial filings.

D. LLM Selection for Financial QA

Qwen 2.5-7B significantly outperforms Mistral 7B (EM: 40.0% vs. 31.8%) and LLaVA 7B (40.0% vs. 27.3%) on this benchmark. LLaVA's 13,602 s latency (vs. 8,097 s for Qwen) reflects the overhead of its vision encoder being invoked for text-only inputs. For practitioners choosing between open-weight models for financial RAG, Qwen 2.5-7B offers the best accuracy-latency tradeoff in our evaluation.

E. Comparison Intent: An Open Problem

All systems achieve 0/5 on Comparison queries before the Round 2 evaluator fix, and at most 3/5 (60.0%) afterward. Multi-hop comparison across documents (e.g., "Which segment grew faster between Apple and Alphabet?") requires reasoning that spans retrieval from multiple independent sources and cannot be resolved in a single-pass architecture. This remains an open research problem for local-only, sub-10B models.

VIII. Conclusions

We presented HierFinRAG and TableTransformerRAG, two complementary architectures for open-domain financial document question answering over corporate 10-K filings. Our key findings are:

1) **Hierarchy + Symbolic Arithmetic are necessary**: HierFinRAG achieves a 13× improvement over the flat baseline (40.0% vs. 3.0% EM).
2) **Visual Table Detection Improves on Complex Documents**: TableTransformerRAG surpasses HierFinRAG (48.2% vs. 40.0% EM) primarily through better table parsing on bookmark-free PDFs.
3) **Two-Pass Extraction**: A second zero-shot extraction call raises Lookup accuracy by 800% (6% → 54%) without any model modification or fine-tuning.
4) **Context Sizing Must Be Intent-Conditioned**: More context hurts accuracy for 7B models; matching context volume to intent type is critical.
5) **Evaluation Integrity**: Gold-context isolation is paramount; leakage can inflate apparent accuracy by >20× (3% real vs. ∼70% inflated).
6) **Local LLMs Are Viable**: All results are achieved via Ollama with open-weight models, demonstrating that commercial API independence is achievable for financial document intelligence.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.

[2] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 6769–6781.

[3] Y. Chen, R. Quach, and H. Ji, "Evaluating open-domain question answering in the era of large language models," *arXiv:2305.06984*, 2023.

[4] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proc. ACM SIGIR*, 2009, pp. 758–759.

[5] Z. Chen, W. Chen, C. Smiley, S. Shah, I. Amin, and C. Sanyal, "FinQA: A dataset of numerical reasoning over financial data," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2021, pp. 3697–3711.

[6] F. Zhu, W. Lei, Y. Huang, C. Wang, S. Zhang, J. Lv, F. Feng, and T. Chua, "TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance," in *Proc. ACL*, 2021, pp. 3277–3287.

[7] Z. Zhu, H. Tan, B. Zhang, and Q. Chen, "HIBRIDS: Attention with hierarchical biases for structure-aware long document summarization," in *Proc. ACL*, 2022, pp. 786–807.

[8] B. Smock, R. Pesala, and R. Abraham, "PubTables-1M: Towards comprehensive table extraction from unstructured documents," in *Proc. IEEE/CVF CVPR*, 2022, pp. 4634–4642.

[9] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. ECCV*, 2020, pp. 213–229.

[10] Q. Team, "Qwen2.5 technical report," *arXiv:2412.15115*, 2025.

[11] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7B," *arXiv:2310.06825*, 2023.

[12] Y. Xiao, C. Lyu, T. Su, J. Wang, Y. Pan, Z. Lian, L. Gui, and K. Feng, "C-pack: Packaged resources to advance general Chinese embedding," *arXiv:2309.07597*, 2023.

[13] Qdrant, "Qdrant: Vector database for the next generation of AI applications," *https://qdrant.tech/*, 2023.